RELATÓRIO TÉCNICO

QUESTÕES TEÓRICAS SOBRE SQL SERVER

Conceitos de consultas, transações, segurança, backup e otimização

Autor	Guilherme Paiva Alves
Instituição	FATEC Osasco
Curso	Desenvolvimento de Software Multiplataforma
Banco de dados	Conecta_Comercio – Microsoft SQL Server

Novembro de 2026

APRESENTAÇÃO

Este relatório apresenta respostas às doze questões teóricas relacionadas ao desenvolvimento e à administração de bancos de dados no Microsoft SQL Server. Os conceitos são explicados com foco em consultas, integridade dos dados, transações, segurança, recuperação e desempenho.

Sempre que possível, os exemplos são relacionados ao banco Conecta_Comercio, utilizado no projeto prático. Nesse contexto, são consideradas tabelas como Clientes, Produtos, Pedidos, ItensPedidos, MovimentosEstoque e Auditorias, além de objetos desenvolvidos durante o projeto, como views, funções, procedimentos armazenados e triggers.

O objetivo é demonstrar não apenas a definição de cada recurso, mas também quando utilizá-lo e quais cuidados devem ser observados em um sistema que registra vendas e controla estoque.

1. Compare View, CTE, subconsulta e tabela temporária: duração, escopo e uso.

Esses recursos permitem organizar e reutilizar consultas, mas diferem principalmente quanto à persistência, ao escopo de utilização e à forma como os resultados são disponibilizados.

Recurso	Duração e escopo	Uso mais adequado
VIEW	É um objeto permanente do banco. Sua definição permanece disponível até ser alterada ou removida. Em uma view comum, os dados não são armazenados como uma cópia: são obtidos das tabelas consultadas quando a view é utilizada.	Centralizar uma consulta reutilizável, simplificar relatórios e oferecer uma interface consistente para acessar dados. Exemplo: vw_ResumoPedidos. [1]
CTE	É um conjunto de resultados nomeado, definido com WITH e válido somente para a instrução imediatamente seguinte. Não é, por padrão, uma tabela materializada.	Dividir uma consulta complexa em etapas mais legíveis, organizar agregações ou fazer consultas recursivas. Exemplo: CTE que consolida vendas por cliente. [2]
Subconsulta	É uma consulta escrita dentro de outra instrução SQL. Seu escopo está ligado à consulta externa e termina com ela.	Aplicar filtros ou cálculos usando o resultado de outra consulta, por exemplo, localizar produtos com preço acima da média. Pode ser escalar, retornar uma lista ou ser correlacionada.
Tabela temporária	É criada em tempdb e armazena temporariamente linhas. A tabela local (#) fica disponível na conexão que a criou; a global (##) pode ser acessada por outras conexões enquanto existir.	Guardar resultados intermediários que serão utilizados em várias instruções, permitir operações sobre esses dados e, quando necessário, criar índices temporários. [3]

Em síntese: a view é um objeto persistente para reutilização; a CTE e a subconsulta existem no contexto de uma única instrução; e a tabela temporária mantém dados intermediários por mais tempo, dentro do seu escopo de sessão ou procedimento. A escolha depende de quantas vezes o resultado será utilizado e de como ele precisa ser manipulado.

2. Qual a diferença entre #local e ##global no SQL Server?

A diferença está no alcance de visibilidade e no ciclo de vida da tabela temporária. Ambas são armazenadas no banco de sistema tempdb, mas possuem escopos distintos. [3]

- #local: começa com um único símbolo # e pode ser acessada somente pela conexão/sessão que a criou. Em geral, é removida quando a conexão termina; quando criada dentro de uma stored procedure, seu escopo termina ao final da execução do procedimento.
- ##global: começa com dois símbolos ## e pode ser acessada por outras conexões da instância enquanto estiver disponível. É removida quando a conexão que a criou termina e não há mais outras tarefas/conexões utilizando a tabela.

Exemplo: uma tabela #ProdutosTemporarios criada em uma janela de consulta do SSMS não estará disponível em outra janela que utilize uma conexão diferente. Já uma tabela ##ProdutosCompartilhados pode ser consultada por outras sessões enquanto permanecer ativa. Por ser compartilhada, uma tabela global exige cuidado com nomes, concorrência e acesso entre usuários.

3. Quando uma stored procedure é preferível a uma function?

Uma stored procedure é preferível quando a necessidade é executar um processo ou uma operação do banco de dados, especialmente quando esse processo modifica dados ou reúne várias etapas que precisam funcionar em conjunto.

Uma procedure pode, por exemplo, validar a existência de um cliente, criar um pedido, inserir seus itens, atualizar o estoque e registrar uma movimentação. Também pode controlar transações com BEGIN TRANSACTION, COMMIT e ROLLBACK, tratar erros com TRY...CATCH, utilizar SQL dinâmico e retornar vários conjuntos de resultados.

Uma function é mais apropriada para encapsular um cálculo ou uma consulta reutilizável, retornando um valor escalar ou uma tabela que pode ser utilizada em uma instrução SELECT. As funções definidas pelo usuário possuem limitações importantes: não podem ser usadas para realizar alterações permanentes no estado do banco e não oferecem o mesmo controle de fluxo e tratamento de erros de uma procedure. [4]

No Conecta_Comercio, a procedure de registro de pedidos é adequada para executar a operação de venda. Já a função fn_CalcularSubtotal é adequada para calcular preço unitário multiplicado pela quantidade, pois retorna um valor sem alterar as tabelas.

4. Por que triggers precisam suportar INSERT/UPDATE/DELETE com várias linhas?

No SQL Server, um trigger DML é disparado por uma instrução que pode afetar uma ou muitas linhas. Por exemplo, um INSERT INTO ... SELECT pode inserir diversos registros de uma só vez, e um UPDATE ou DELETE pode atingir centenas de linhas em uma única execução. O trigger é disparado por instrução, não uma vez para cada linha afetada. [5]

Para tratar corretamente essas situações, o código deve ser orientado a conjuntos e utilizar as tabelas lógicas inserted e deleted. A tabela inserted contém os novos valores inseridos ou atualizados; a tabela deleted contém os valores removidos ou os valores anteriores a um UPDATE. Em um UPDATE, as duas tabelas permitem comparar o estado anterior e o posterior. [6]

Se o trigger considerar que inserted contém apenas uma linha, poderá processar somente parte dos registros, atribuir um valor inesperado a uma variável ou produzir totais incorretos. Por isso, deve utilizar operações como JOIN, GROUP BY e agregações para tratar o conjunto completo.

No projeto, a trigger trg_AtualizarEstoque agrupa as linhas de inserted por id_produto e soma as quantidades antes de atualizar o estoque. A trigger trg_AuditarProdutos compara inserted e deleted para registrar as alterações. Esse desenho permite que uma única instrução que afete vários produtos seja processada corretamente.

5. Explique os quatro princípios ACID no contexto de uma venda.

ACID representa quatro propriedades que ajudam a garantir que uma transação seja confiável: Atomicidade, Consistência, Isolamento e Durabilidade. Uma transação de venda pode envolver a criação do pedido, a inclusão de itens, a baixa do estoque e o registro da saída em MovimentosEstoque. [7]

Atomicidade — todas as etapas da venda são concluídas ou nenhuma delas permanece registrada. Se a inclusão de um item falhar por falta de estoque, por exemplo, a transação deve ser desfeita para não deixar um pedido parcialmente gravado.

Consistência — a transação deve respeitar as regras de integridade antes e depois da execução. No projeto, isso inclui chaves estrangeiras válidas, quantidade maior que zero, situação permitida do pedido e estoque não negativo. A venda deve levar o banco de um estado válido para outro estado válido.

Isolamento — transações concorrentes não devem interferir de forma a produzir resultados incorretos. Se duas vendas tentarem consumir o estoque do mesmo produto ao mesmo tempo, o controle de concorrência deve impedir que ambas utilizem a mesma quantidade disponível e deixem o estoque inconsistente.

Durabilidade — depois que a transação é confirmada com COMMIT, suas alterações devem permanecer registradas mesmo diante de uma falha do processo ou do servidor, conforme os mecanismos de log e recuperação do SQL Server.

Assim, uma operação de venda deve agrupar as etapas relacionadas em uma transação e utilizar COMMIT somente quando todas forem concluídas com sucesso; em caso de erro, deve executar ROLLBACK.

6. O que diferencia LOGIN, USER e ROLE?

LOGIN, USER e ROLE são elementos de segurança com responsabilidades diferentes no SQL Server:

- LOGIN: identidade criada no nível da instância do SQL Server. É utilizada na autenticação, ou seja, para comprovar que uma pessoa ou aplicação pode se conectar ao mecanismo de banco de dados.
- USER: identidade no nível de um banco específico. Normalmente é mapeada para um login e recebe permissões sobre objetos daquele banco, como tabelas, views e procedures.
- ROLE: grupo de segurança utilizado para reunir usuários e atribuir permissões de forma centralizada. Em vez de conceder SELECT individualmente a cada usuário, pode-se conceder a permissão a uma role e adicionar os usuários como membros.

No exercício, login_conecta_consulta representa o login da instância; user_conecta_consulta é o usuário mapeado no banco Conecta_Comercio; e role_consulta agrupa esse usuário para receber permissões de leitura. Essa separação facilita administrar o acesso e aplicar o princípio do menor privilégio. [8]

7. Diferencie GRANT, REVOKE e DENY.

Esses comandos controlam as permissões de segurança. Podem ser aplicados a usuários, roles e outros principais de segurança, conforme o escopo do objeto protegido. [9]

- GRANT: concede uma permissão. Exemplo: permitir que a role role_consulta leia os objetos do schema dbo.
- REVOKE: remove uma concessão ou negação explícita anteriormente registrada. Não significa necessariamente bloquear o acesso; após o REVOKE, o resultado depende de outras permissões herdadas ou concedidas por outros meios.
- DENY: registra uma negação explícita de uma permissão. Em regra, uma negação explícita prevalece sobre uma concessão comum, ressalvadas exceções previstas pelo modelo de segurança, como permissões de funções administrativas e propriedade de objetos.

Exemplos no banco Conecta_Comercio:

```
GRANT SELECT ON SCHEMA::dbo TO role_consulta;
REVOKE SELECT ON SCHEMA::dbo FROM role_consulta;
DENY DELETE ON dbo.Produtos TO user_conecta_consulta;
```

Portanto, GRANT permite, DENY proíbe explicitamente e REVOKE retira uma configuração explícita para que a permissão volte a ser determinada pelas demais regras aplicáveis. É importante verificar a associação a roles e as permissões herdadas antes de concluir que um usuário perdeu ou recebeu acesso.

8. Como verificar se um backup é recuperável?

A verificação de um backup deve ir além de confirmar que o arquivo .bak existe. O procedimento recomendado combina validação do conjunto de backup com um teste real de restauração.

1. Gerar o backup com opções de verificação, como CHECKSUM, e armazená-lo em um local protegido, com espaço disponível e permissões adequadas para a conta de serviço do SQL Server.
2. Executar RESTORE VERIFYONLY para verificar se o conjunto de backup está completo e se o conteúdo do backup pode ser lido. Essa instrução não restaura o banco e não verifica integralmente a estrutura lógica dos dados. [10]
3. Realizar uma restauração de teste em outro nome de banco ou em um ambiente separado, evitando sobrescrever a base de produção.
4. Após a restauração, executar DBCC CHECKDB e validar tabelas, registros e operações essenciais da aplicação. A recuperação só foi efetivamente testada quando o banco restaurado pode ser aberto e utilizado de forma consistente.

No exercício do Conecta_Comercio, RESTORE VERIFYONLY é uma etapa útil, mas não substitui a restauração do arquivo FULL em um banco de teste. Também é importante documentar a frequência dos backups, o local de armazenamento, a retenção e quem é responsável por testar a recuperação. Um backup que nunca foi restaurado em teste não oferece a mesma confiança de um processo de recuperação validado.

9. O que é índice nonclustered e qual seu custo de manutenção?

Um índice nonclustered é uma estrutura separada das páginas de dados da tabela, organizada pelas colunas definidas como chave do índice. Suas folhas armazenam os valores das chaves e um localizador que permite encontrar a linha correspondente na tabela ou no índice clustered. Também é possível incluir colunas adicionais com INCLUDE para que determinadas consultas encontrem no próprio índice todos os dados de que precisam. [11]

Esse tipo de índice pode acelerar filtros, junções, ordenações e consultas que acessam frequentemente determinadas colunas. Por exemplo, um índice em Pedidos(id_cliente, data_pedido) pode ajudar consultas que filtram pedidos por cliente e período.

O benefício de leitura tem um custo de manutenção:

- INSERT: cada novo registro também precisa ser representado nos índices aplicáveis.
- UPDATE: quando uma coluna que participa da chave ou das colunas incluídas é alterada, o índice pode precisar ser atualizado.
- DELETE: as entradas correspondentes precisam ser removidas das estruturas de índice.
- Armazenamento e manutenção: cada índice ocupa espaço e pode exigir atualização de estatísticas, reorganização ou reconstrução conforme o uso e a fragmentação.

Por isso, criar índices em excesso pode aumentar o custo das gravações e do armazenamento. O ideal é criar índices com base nas consultas mais relevantes, verificar se são realmente utilizados e evitar incluir colunas desnecessárias.

10. Explique plano de execução, SEEK, SCAN e SARGability.

O plano de execução mostra como o otimizador do SQL Server pretende executar uma consulta ou como ela foi executada. Ele apresenta operadores como buscas e varreduras de índices, junções, ordenações e agregações, além de estimativas de custo e, no plano real, informações observadas durante a execução. A análise do plano ajuda a identificar operações caras, estimativas incorretas e acessos desnecessários aos dados. [12]

Index Seek — busca: utiliza a estrutura de um índice para localizar diretamente uma chave ou faixa de valores. Costuma ser interessante quando o filtro seleciona uma parcela relativamente pequena dos registros.

Index Scan ou Table Scan — varredura: percorre as entradas de um índice ou as páginas da tabela para avaliar os registros. Não é necessariamente um problema: pode ser a melhor opção quando a consulta precisa de grande parte dos dados ou quando a tabela é pequena.

SARGability — capacidade de um predicado ser utilizado pelo mecanismo de busca de índices. Uma condição SARGable permite que o otimizador avalie estratégias como Index Seek. Aplicar uma função diretamente sobre a coluna filtrada ou converter a coluna a cada comparação pode dificultar o uso eficiente do índice. [13]

Exemplo de filtro geralmente mais favorável a um índice em data_pedido:

```
WHERE data_pedido >= '20260101'
  AND data_pedido <  '20270101'
```

Uma forma menos favorável seria aplicar YEAR(data_pedido) = 2026 diretamente na coluna. Nesse caso, o mecanismo pode precisar calcular YEAR para muitas linhas antes de avaliar o filtro. A reescrita como intervalo permite comparar diretamente a coluna com limites de data.

A escolha entre SEEK e SCAN depende da seletividade do filtro, do volume de dados, das estatísticas e das colunas disponíveis nos índices. Portanto, não se deve concluir que todo SCAN é ruim ou que todo SEEK é automaticamente mais rápido.

11. Qual o risco de usar NOLOCK para relatórios financeiros?

A dica NOLOCK permite leituras no nível READ UNCOMMITTED. Com isso, uma consulta pode ler alterações feitas por outra transação antes de elas serem confirmadas. Se a transação concorrente executar ROLLBACK, o relatório poderá ter considerado valores que nunca se tornaram dados definitivos. Esse fenômeno é conhecido como leitura suja. [14]

Em relatórios financeiros, o risco é elevado: totais de vendas, saldos, valores de pedidos e movimentações de estoque podem ficar incorretos. Além de leituras sujas, a concorrência pode causar leituras duplicadas ou linhas não observadas durante determinadas varreduras, produzindo agregações inconsistentes.

NOLOCK também não significa ausência absoluta de bloqueios: consultas ainda podem adquirir bloqueios de estabilidade de esquema. Assim, a dica não deve ser tratada como solução universal para lentidão.

Para relatórios que exigem confiabilidade, deve-se utilizar um nível de isolamento apropriado, normalmente READ COMMITTED ou, quando adequado e configurado, isolamento baseado em versionamento de linhas, como READ COMMITTED SNAPSHOT ou SNAPSHOT. A escolha deve considerar consistência, concorrência e requisitos da aplicação.

12. Por que a otimização deve preservar o resultado funcional?

O objetivo da otimização é reduzir o custo de execução sem alterar o significado da consulta ou as regras de negócio. Uma consulta mais rápida não representa uma melhoria válida se deixar de retornar pedidos que deveriam aparecer, duplicar linhas, excluir valores nulos relevantes ou calcular totais diferentes.

Ao comparar a versão original e a versão otimizada, é necessário preservar, conforme o requisito da consulta:

- os mesmos registros e critérios de filtragem;
- a cardinalidade e a ocorrência de duplicidades, respeitando os relacionamentos e as agregações;
- os mesmos cálculos, tipos numéricos, precisão e tratamento de valores NULL;
- os mesmos limites de datas e condições de negócio;
- a ordenação quando ela for explicitamente exigida por ORDER BY.

No Conecta_Comercio, por exemplo, adicionar um índice para acelerar a consulta de pedidos não deve alterar a quantidade de pedidos retornados nem o valor total das vendas. A validação deve comparar os resultados antes e depois, além de analisar plano de execução, leituras lógicas e tempo. A otimização só é bem-sucedida quando melhora o desempenho e mantém a correção funcional.

CONSIDERAÇÕES FINAIS

As questões demonstram que o desenvolvimento de banco de dados envolve mais do que escrever consultas SQL. É necessário escolher estruturas adequadas para organizar consultas, compreender o ciclo de vida dos dados temporários, implementar triggers orientadas a conjuntos, controlar transações, administrar permissões e validar estratégias de backup.

Também ficou evidente que desempenho e confiabilidade devem ser tratados em conjunto. Índices e reescritas de consultas podem reduzir o custo de leitura, mas precisam ser avaliados quanto ao custo de escrita e, principalmente, sem alterar os resultados esperados. Da mesma forma, mecanismos como transações ACID, permissões bem definidas e testes de restauração protegem a integridade e a disponibilidade do banco Conecta_Comercio.

REFERÊNCIAS

Documentação oficial do Microsoft Learn consultada para fundamentar os conceitos deste relatório. Acesso em: 10 nov. 2026.

[1] Microsoft. Views (SQL Server). Acessar documentação

[2] Microsoft. WITH common_table_expression (Transact-SQL). Acessar documentação

[3] Microsoft. Tables – Temporary tables (SQL Server). Acessar documentação

[4] Microsoft. Create User-Defined Functions (Database Engine). Acessar documentação

[5] Microsoft. Create DML Triggers to Handle Multiple Rows of Data. Acessar documentação

[6] Microsoft. Use the inserted and deleted Tables. Acessar documentação

[7] Microsoft. Transaction Locking and Row Versioning Guide. Acessar documentação

[8] Microsoft. Get Started with Database Engine Permissions. Acessar documentação

[9] Microsoft. Database-Level Roles / Permissions. Acessar documentação

[10] Microsoft. RESTORE VERIFYONLY (Transact-SQL). Acessar documentação

[11] Microsoft. Index Architecture and Design Guide. Acessar documentação

[12] Microsoft. Logical and Physical Showplan Operators Reference. Acessar documentação

[13] Microsoft. Troubleshoot High CPU Usage Issues in SQL Server – SARGability. Acessar documentação

[14] Microsoft. Table Hints (Transact-SQL). Acessar documentação